

No Permanent Champion: A Deployment Study of SGLang and vLLM across Dense, Ultra-Sparse MoE, and KDA-Hybrid Architectures on H200 GPUs

Chao-Chun Chuang^{1,*} and Po-Hsiang Lin²

¹National Center for High-performance Computing (NCHC),
National Institutes of Applied Research (NIAR), Taiwan

²Kaohsiung Veterans General Hospital, Kaohsiung, Taiwan

* Corresponding author: c00cjz00@nchc.org.tw

September 2026

Abstract

We operate a multi-model API service on an HPC cluster—a LiteLLM gateway fronting Slurm-managed SGLang and vLLM engines on NVIDIA H200 GPUs—and each new model it onboards raises the same question: which serving framework will run this checkpoint faster? We encoded an A/B comparison procedure as versioned procedures (“skills”) that an LLM-based agent executed under human approval: scaffolding both frameworks’ engines from official per-model documentation, deploying them under traffic-isolated aliases, and driving the same single-burst 500–1,000-user workload at each, with aggregate output-token counts compared per run. Applied to three 2026 models—a 27B dense hybrid, a 180B-total/6B-active ultra-sparse MoE, and a 288-expert Kimi Delta Attention (KDA) hybrid MoE—the observed leader changed each time: SGLang led by 6.3% (near parity) on the dense model, vLLM by $2.33\times$ on the MoE, and SGLang by 28% on the KDA hybrid. Speculative decoding on the same checkpoints reduced measured throughput in five of six framework–model pairs (single-run burst measurements) and cut one run’s success rate to 85.6%; disabling it on the production engine gained 17–20% the same day. An agent-assisted layered diagnosis, encoded as a symptom-to-cause procedure, raised mixed 1,500-user success from 67.9% to 100% by correcting file-descriptor limits. These observations indicate that framework choice and speculative decoding should be measured per model rather than inherited as defaults; the procedure is a versioned artifact of our repository.

1 Introduction

The throughput and latency of a deployed large language model (LLM) depend not only on the model and the accelerator, but on the serving framework that schedules batches, manages KV-cache memory, and executes attention and mixture-of-experts (MoE) kernels. Among the open-source frameworks used for self-hosted deployment, vLLM is the most widely adopted in practice and SGLang is among its principal alternatives [1]. vLLM was introduced with PagedAttention for efficient KV-cache memory management [2], and SGLang with a RadixAttention prefix cache and a structured-generation runtime [3]. Both systems evolve quickly, add support for new model architectures as they appear, and make different implementation trade-offs for the same architecture. Practitioners therefore face a recurring question that vendor documentation does not answer: *for this specific model, on this hardware, which framework will serve it faster?*

This question is not academic for us. We operate a multi-model API service for the users of an HPC cluster: a LiteLLM gateway fronting Slurm-managed inference engines—SGLang and vLLM containers among them—on NVIDIA H200 GPUs, with per-engine lifecycle management, versioned container images, and declarative routing that lets a candidate engine receive production-shaped traffic under isolated aliases before it claims any production name. Every time the center onboards a new open-weight model, the platform forces a concrete framework decision, and the wrong one is expensive: as this paper shows, the cost of choosing wrongly reaches $2.3\times$ in throughput—larger than any tuning lever we measured, including correcting an outright misconfiguration (+48% on the dense engine’s earlier settings). Manual evaluation is slow and error-prone (framework-specific flags, memory budgets, and validation steps differ), so we delegated it to an agent: the evaluation procedure—engine scaffolding from each framework’s official per-model documentation, dual-framework deployment, paired benchmarking, verdict recording, and production traffic switching—was encoded as declarative, versioned procedures (which we call *skills*) and executed by an LLM-based coding agent under human approval. Each framework decision in this study was completed within a day; we did not measure the agent’s unattended time or the labor it saved.

Published comparisons between serving systems do exist: an empirical evaluation of vLLM against HuggingFace TGI on LLaMA-2 models is a recent example [4], surveys consolidate the serving-systems literature [5], and a study of open-source deployments documents which frameworks practitioners actually adopt [1]. Same-checkpoint comparisons of multiple backends have quantified accuracy fidelity [6], and SiliconBench evaluates nine Apple Silicon serving engines on unified-memory desktops, using CUDA vLLM and SGLang as references [7]; neither addresses throughput at hundreds to a thousand concurrent requests on HPC GPUs, and none evaluates ultra-sparse MoE or KDA hybrids. Separately, the fragility of speculative decoding under batched serving is not merely a vendor warning: goodput analyses and adaptive speculation control have formalized when drafting helps and when it hurts [8, 9]. What remains unmeasured is how these framework and speculation choices behave on the newest architectures—with checkpoint-integrated multi-token prediction (MTP) heads rather than external draft models—under high-concurrency burst load.

The present paper reports what this agent-assisted workflow found, together with the workflow itself. We make four contributions:

1. **An agent-assisted evaluation workflow** (Sections 3–4): a paired A/B procedure—both frameworks serving the *same local checkpoint* on the same GPU type under the *same client workload*, engine parameters taken from official documentation, and aggregate output-token counts compared and disclosed pair by pair—encoded as versioned skills that an LLM-based agent executed under human approval, from scaffolding the candidate engine to switching production traffic when the verdict is in.
2. **Three per-model observations** (Section 5): SGLang 0.5.20 led by 6.3% (near parity) on a dense 27B hybrid; vLLM 0.29.1rc1 led by $2.33\times$ on a 180B/6B-active ultra-sparse MoE; SGLang led by 28% on a 306 GB FP8 KDA model—indicating that the leading framework is model-specific, with differences large enough (up to $2.3\times$) to outweigh the other tuning levers we measured.
3. **A negative result for speculative decoding under load**: across the three models and two frameworks (six framework–model pairs), speculative decoding reduced aggregate throughput in five cases (by 15–46%) and, in the worst case, degraded the request success rate to 85.6% through timeouts; one verdict was applied to production the same day (+17–20%).
4. **A production platform description and reliability analysis** (Sections 3 and 5.5): a LiteLLM-based multi-engine gateway with Slurm-managed engine lifecycle, declarative

production/staging traffic switching, and versioned container images, together with a layered diagnosis—also agent-assisted, from an encoded symptom-to-cause procedure—that traced mixed-load failures (67.9% $\rightarrow$ 91.8% $\rightarrow$ 100%) to file-descriptor limits in the client shell and per-connection sshd, not to GPU engines or the gateway.

All experiments were run in September 2026 on the platform described in Section 3; raw per-run statistics are released with the platform repository.

2 Related Work

Serving systems. Iteration-level scheduling and continuous batching were introduced by Orca [10] and adopted by vLLM, whose PagedAttention brought operating-system-style page management to KV-cache memory [2]. SGLang contributed RadixAttention for automatic prefix reuse and an efficient runtime for structured generation programs [3]. Chunked prefill, proposed in SARATHI, mitigates the interference of long prefills with decodes [11], and DistServe shows that disaggregating prefill from decoding improves goodput under latency constraints [12]. Kernel libraries such as FlashInfer supply portable attention and decoding kernels that both frameworks integrate [13]. Our work does not propose a new serving mechanism; it measures two mature frameworks on a production platform and finds that their relative standing differed across the three models tested.

Model architectures under test. The three models we evaluate span the current design space of efficient open-weight LLMs. Qwen3.8-27B continues the dense hybrid line of Qwen3 [14], combining gated delta net (GDN) linear attention with full attention. Qwen3.8-Flash-Next is an ultra-sparse MoE (180B total, 6B active parameters, 512 experts) with a large N-gram embedding table, in the lineage of DeepSeek-V2/V3’s economical MoE and multi-head latent attention (MLA) designs [15, 16]. GLM-5.3-Flash continues the GLM agentic line [17] as a native-FP8, 288-expert MoE (8 experts active per token, one shared expert, first three layers dense) whose 45-layer attention stack interleaves, three-to-one, 34 Kimi Delta Attention (KDA) linear-attention layers—the architecture introduced by Kimi Linear [18]—with 11 DeepSeek Sparse Attention (DSA) layers, plus a 1M-token context window; its closest published relatives include Kimi K2’s large sparse-MoE design [19], and serving-side research on KDA-style hybrids is already emerging—recurrent-state compression (DASC) [20], decay-aware mixed-precision state quantization (DAMP) [21], and switchable per-layer mixer placements, including KDA, served from one supernet checkpoint (Super April) [22]. The architectural diversity of this trio is a plausible reason framework rankings might not transfer across models—a hypothesis our results support but cannot isolate, given one model per architecture.

Speculative decoding. Speculative sampling accelerates autoregressive decoding by drafting multiple tokens with a cheap proposal mechanism and verifying them in parallel with the target model [23]. Proposal mechanisms include additional decoding heads (Medusa [24]), feature-level draft models with uncertainty handling (EAGLE [25]), and the checkpoint-integrated MTP heads of DeepSeek-V3 [15] and its successors, which both frameworks in this study expose as NEXTN/MTP options. The published literature evaluates speculative decoding primarily as a *latency* optimization at low request concurrency [23–25], but its behavior under batched serving has been studied: goodput analyses observe that speculation can degrade serving performance when applied without regard to load, and adaptive systems such as TurboSpec close the loop by enabling drafting only when it pays [8]; AdaServe customizes speculation to per-request SLOs [9]. Our contribution is narrower and complementary: a systematic measurement of *checkpoint-integrated* MTP/NEXTN heads across two frameworks and three architectures under high-concurrency burst load, where we find drafting to be a net cost in five of six cases.

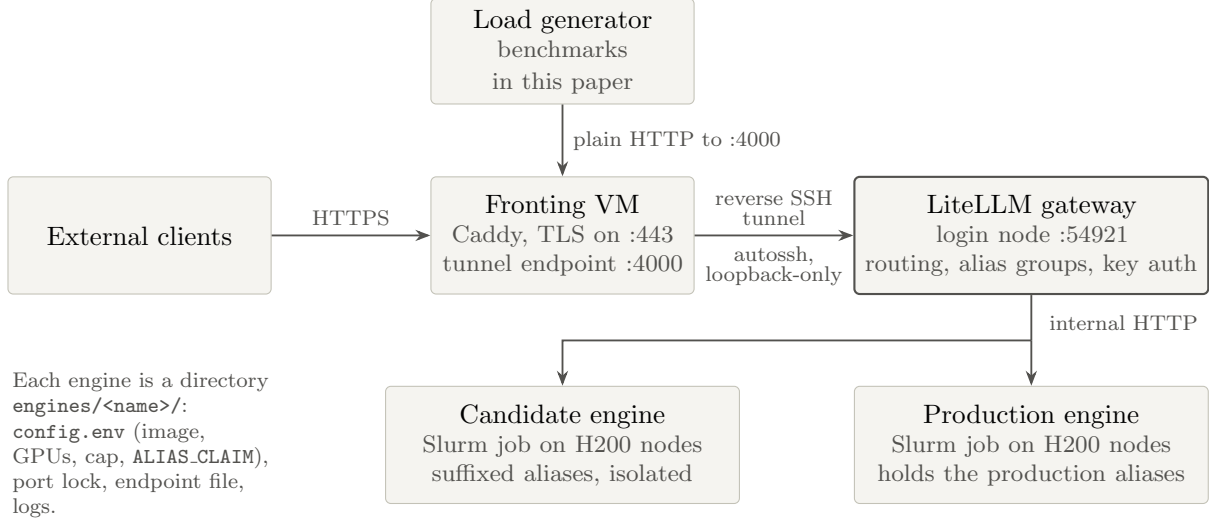


Figure 1: Request path from external clients through the gateway to engine jobs. A candidate engine registers only suffixed aliases, so it receives benchmark traffic without claiming production names; switching which engine holds the production aliases is a two-line `ALIAS_CLAIM` edit plus a gateway restart (Section 3). The benchmarks in this paper entered at the tunnel’s plain-HTTP endpoint on the VM, so they exercised the tunnel, gateway, and engines but not the TLS front end.

Empirical comparisons of serving frameworks. A comparative performance study of vLLM and HuggingFace TGI on LLaMA-2 models (7B–70B) found framework-dependent trade-offs in throughput, latency, and memory [4]; a survey consolidates the broader serving-systems literature [5]; and a large-scale analysis of open-source systems documents which serving frameworks and methods are adopted in practice [1]. The Silent Hyperparameter holds weights and decoding fixed across five backends but measures accuracy reproducibility rather than throughput [6]; SiliconBench evaluates nine Apple Silicon serving engines on unified-memory desktop hardware, using CUDA vLLM and SGLang as references, with fidelity as a first-class metric [7]; and AutoTuneBench has agent systems auto-tune both engines under a measurement protocol stricter than ours (paired seeds, cross-run CV at most 5%) [26]. Relative to these, our study differs in target (high-concurrency burst throughput on a production HPC platform with H200 GPUs), in scope (three 2026 hybrid architectures, including—to our knowledge—the first cross-framework serving comparison of a KDA-hybrid model [18]), and in the object of automation: prior agent work tunes engine configurations [26] or generates bespoke serving stacks from scratch [27], whereas our skills guide an agent through the production lifecycle of an existing platform, from deployment behind a live gateway to production traffic switching.

3 The Platform: A Production LLM Serving Stack for HPC Clusters

All experiments ran on the serving platform we operate for internal users of an HPC cluster (NCHC). This section documents the platform both to establish experimental validity (Section 4) and because the platform design itself—engine lifecycle, traffic switching, and image governance—encodes operational lessons that we believe are reusable. Figure 1 outlines the request path; Table 1 summarizes the components.

Three design decisions deserve emphasis because they made the A/B protocol of Section 4 cheap

Component	Implementation	Notes
Gateway	LiteLLM proxy, login node, port 54921	5.2% CPU / 333 MB (observed during the 1,000-user load test)
Authentication	Master key + per-user virtual keys (<code>key_tool.py</code>)	JSON key store with flock and atomic writes; no database; per-key RPM/TPM limits enforced in memory
Engine lifecycle	<code>start_models.sh</code> / <code>stop_models.sh</code> over Slurm	Idempotent dispatch, port locks, endpoint readiness files, two-stage validation (<code>validate_engine.sh</code>)
Routing config	<code>generate_runtime_config.py</code>	Static portal models + live probing of engine endpoints; alias groups with production/isolated claims
Traffic switching	ALIAS_CLAIM in each engine’s <code>config.env</code>	Production alias set vs. suffixed aliases; switching frameworks = edit two lines + gateway restart (~10 s), engines keep running
Container images	Versioned Apptainer SIF files (<code>sglang.0.5.20.sif</code> , <code>vllm.0.29.1rc1.sif</code> , <code>vllm_glm53-flash.sif</code>), dedicated	Version-named; the pull script refuses to overwrite an existing SIF unless forced, and no SIF used in this study was overwritten; dedicated vendor builds used when a model requires them; no content digests recorded
External access	Caddy TLS + autossh reverse tunnel, plus SSH tunnel and web-proxy paths	Loopback-only tunnel binding; IP allowlists at the fronting VM

Table 1: Platform components.

to run and safe for production traffic. *First*, engines are self-contained directories (`engines/<name>/`) whose `config.env` declares the container image, model path, port, GPU topology, concurrency cap, speculative-decoding settings, and alias claims; starting, stopping, and validating an engine is a single command each, and an engine’s failure cannot affect other engines because Slurm isolates it. *Second*, the gateway’s routing configuration is generated from the *live* set of engine endpoints, so bringing a candidate engine up under suffixed aliases lets it receive production-shaped traffic for validation without claiming the production names; promoting it (or rolling back) is a two-line configuration edit and a ten-second gateway restart. *Third*, container images are version-named SIF files that the pull script refuses to overwrite by default (none used here was overwritten), with a policy of using a vendor’s *dedicated* build when one exists for a given model; this policy was validated during the study when a generic nightly vLLM build failed on the KDA model (kernel assertion plus an autotuning hang) while the dedicated build ran correctly (Section 5.3).

Evaluation automation: skills and the agent. The A/B protocol of Section 4 is not a manual runbook. It is encoded as declarative, versioned procedures—*skills*, in the sense of composable packages of instructions and resources that agents load on demand [28]—which an LLM-based coding agent with shell access executes under human approval. Three skills carry the methodology. The *model-onboarding* skill governs how any new model enters the platform: it is official-documentation-first (the agent must locate the vendor’s per-model Cookbook or Recipe pages and derive engine flags from them, rather than inventing configurations), recommends dual-framework deployment whenever both frameworks support a model (in this study, each model was scaffolded and validated on both SGLang and vLLM under isolated aliases), and encodes the image-version decision flow (versioned names, dedicated builds preferred, “once supported $\neq$ always

Pipeline stage	Agent-executed	Human decision
Locate official per-model documentation	✓	—
Scaffold + validate both frameworks’ engines	✓	Framework releases to pin
Deploy under traffic-isolated aliases	✓	—
Run paired benchmark, compare output-token counts	✓	Workload shape
Record verdict (validated-configuration registry)	✓	Interpretation
Switch production traffic / roll back	✓	Go/no-go
Failure diagnosis during any stage	✓ (from encoded symptom→cause procedures)	—

Table 2: Division of labor in the agent-assisted evaluation workflow. Skills are version-controlled artifacts of the repository, so the procedure is documented as executable instructions; re-running a decision still requires the same models, images, and hardware.

supported”). The *concurrency-troubleshooting* skill encodes the layered diagnosis of Section 5.5 as a symptom→cause→fix table, and the *debug-journaling* skill requires that every failure and fix be recorded with its evidence—Tables 4–9 of this paper are extracted from those records. All procedures are released as public, versioned artifacts of the platform repository (`.agents/skills/`), making the procedure itself a citable artifact rather than prose. Table 2 summarizes the division of labor: the agent performs the mechanical chain (scaffold both engines, pull versioned images, run two-stage validation, deploy under isolated aliases, execute the same client workload, compare aggregate output-token counts, compare throughput, record the verdict in a registry of validated configurations, and switch production traffic by a two-line edit), while humans retain the decisions (which model to onboard, which framework releases to pin, and how to weigh verdicts).

4 Methodology

The evaluation asked two questions per model: (i) which framework serves it faster under a high-concurrency burst, and (ii) does enabling the checkpoint’s speculative-decoding head change the answer or the absolute throughput. We answer both with paired A/B runs whose conditions are summarized in Table 3. The procedure was executed by the LLM-based agent described in Section 3, following the versioned skills; the human authors made the release decisions (which model, which framework versions to pin) and accepted or rejected verdicts. This execution model matters for validity: every engine flag in every run is traceable to the official documentation the skill requires the agent to cite, and each run’s statistics are machine-recorded artifacts rather than transcriptions, except the initial mixed-run success rate of Section 5.5, which is taken from the operations log.

Fairness controls. Each pair of runs used (a) the same local checkpoint directory (bit-identical weights, no downloads), (b) the same GPU type in the same time window, with runs executed back-to-back (scheduler placement put some pairs on different nodes of the same cluster; the job-to-node placement of every engine active in each comparison window, failed attempts included, is tabulated in `benchmarks/results/ab_run_node_mapping.csv`), (c) concurrency caps set equal

Condition	Dense comparison	Ultra-sparse MoE comparison	KDA comparison
Model (same checkpoint both frameworks)	Qwen3.8-27B (BF16, 52 GB)	Qwen3.8-Flash-Next (FP8, 173 GB; 180B/6B active)	GLM-5.3-Flash (FP8, 306 GB)
Architecture	Dense hybrid: GDN + full attention	Ultra-sparse MoE: 512 experts + 51B N-gram embedding	KDA linear attention + DSA, 288-expert MoE (8 active), 1M context
GPUs per engine	1× H200	4× H200 (TP4+EP4)	8× H200 (TP8+EP8 / TEP8)
Frameworks	SGLang 0.5.20 vs vLLM 0.29.1rc1	SGLang 0.5.20 vs vLLM 0.29.1rc1	SGLang 0.5.20 vs vLLM 0.28.1rc1 (dedicated GLM build)
Engine parameters	Official docs: SGLang Cookbook vs vLLM Recipe (per-architecture pages)	idem; <code>--max-num-seqs 256</code> per Recipe; SGLang cap aligned to 256	idem; BF16 KV cache (FP8 KV unsupported on Hopper for this model, per Recipe)
Concurrency cap	128 both sides	256 both sides	128 both sides
Workload	500 concurrent users × max_tokens 500	1,000 users × 800	1,000 users × 800
Output tokens (aggregate)	207,832 vs 207,638 (±0.1%)	516k–525k (R1 pair ±0.8%; R2 pair 1.46%, disclosed)	732k–734k (R1); R2 vLLM censored at 623k by timeouts

Table 3: A/B conditions. “R1” denotes the plain configuration (no speculative decoding, the production recommendation); “R2” denotes speculative decoding enabled (the checkpoint’s integrated MTP/NEXTN head, with each framework’s recommended step/top-k settings). Architecture classes: `Qwen3.5ForConditionalGeneration` (dense), `Qwen4ExpForConditionalGeneration` (ultra-sparse MoE), `Glrm5NextForConditionalGeneration` (KDA hybrid). TP/EP: tensor/expert parallelism; TEP: combined tensor–expert parallelism.

on both sides, and (d) engine startup parameters taken verbatim from each framework’s official per-model documentation rather than tuned by us; where the two frameworks’ defaults conflicted (e.g., an embedding-offload strategy on the MoE model), we report both variants (Section 5.2). Aggregate output-token counts were compared per run; this check does not establish equal per-request computational work or equal output quality. R1 pairs fell within ± 0.1 –0.8%. Two R2 pairs deviated further and are reported with that disclosure rather than discarded: the dense pair differed by 1.01% and the MoE pair by 1.46%; the KDA vLLM R2 run counted 15% fewer output tokens than its pair (623k vs 733k) because 144 requests hit the client timeout and are not counted, so its throughput covers successful responses only and its wall time ends when the last requests timed out (301.5 s)—a censored measurement we flag wherever it is cited. Success rate, aggregate output throughput (tokens/s), P50/P95 latency, and total wall time were collected by the same client harness, an httpx-based load generator (`stress.test.py`) that drives a fixed pool of five short technical-QA prompts (cycled across requests at temperature 0.7) from the fronting VM through the reverse tunnel and the gateway to the engine rather than bypassing the gateway; it targets the tunnel’s plain-HTTP endpoint on the VM, so the TLS front end was not in the measured path.

Metrics. We report *aggregate* throughput (total output tokens divided by wall time), *success rate* (completed requests $\div$ total), and *tail latency* (P95). We treat a configuration as production-eligible only at 100% success; one R2 configuration failed this bar (Section 5.3).

Metric	SGLang 0.5.20 (R1)	vLLM 0.29.1rc1 (R1)
Success	500/500 (100%)	500/500 (100%)
Aggregate throughput	3,823 tok/s	3,597 tok/s
P95 latency	51.8 s	54.6 s
Wall time	54.4 s	57.7 s

Table 4: Dense comparison, R1 (plain). SGLang +6.3% throughput, −5.1% P95.

Metric	SGLang EAGLE (3/1/4)	vLLM MTP×3
Concurrency cap	48 (auto-reset by SGLang)	128
Aggregate throughput	2,306 tok/s (−40%)	3,034 tok/s (−16%)
P95 latency	87.2 s	64.4 s

Table 5: Dense comparison, R2 (speculative decoding). Percentages relative to each framework’s own R1. EAGLE settings are steps/draft/tokens.

Limitations of protocol. Each configuration was measured once as a single burst: all requests are issued at once and the run ends when the queue drains. Without repeated runs we cannot estimate run-to-run variance, so every margin reported here is a single observation rather than an established effect; small differences, such as the dense +6.3%, are read as near parity. All measurements are single-instance (one engine deployment per framework per model); the engines’ Slurm allocations per comparison window are recorded in `benchmarks/results/ab_run_node_mapping.csv`.

5 Results

5.1 Dense hybrid: SGLang +6.3%

On the 27B dense hybrid, both frameworks completed all 500 requests successfully; SGLang had higher measured throughput and lower P95 latency and wall time (Table 4).

Enabling the checkpoint’s integrated MTP head hurt both frameworks, but unevenly (Table 5). SGLang automatically resets the concurrency cap from 128 to 48 when speculative decoding is enabled; the engine log reports this as a reset for speculative decoding, and our engine-configuration notes attribute it to the draft path’s memory footprint (draft weights plus a 13.8 GB intermediate-state cache for the GDN blocks). Aggregate throughput fell by 40% relative to its own R1—a figure that combines the cap reduction with the speculation overhead itself, since we did not run a cap-48 plain control to separate the two. vLLM’s MTP shares the head weights within the same checkpoint and kept cap 128, yet still lost 16%, consistent with verification FLOPs competing with useful decode work at these batch sizes; we did not profile GPU utilization to confirm the mechanism. Neither R2 configuration beat its R1 counterpart.

5.2 Ultra-sparse MoE: vLLM 2.33×

On the ultra-sparse MoE, the ranking reversed (Table 6).

The plain vLLM configuration was 2.33× faster than plain SGLang, and 2.01× faster than SGLang’s *best* configuration. Notably, the same integrated draft head moved the two frameworks in opposite directions: SGLang’s NEXTN gained 16%—consistent with the model’s low 6B-active decode cost making draft-and-verify cheap—while vLLM’s MTP lost 46%, matching the vendor

Configuration	SGLang 0.5.20	vLLM 0.29.1rc1
R1 aggregate throughput	3,383 tok/s ¹	7,886 tok/s
R1 P95 latency	150.6 s	63.2 s
R2 (speculative)	3,925 tok/s (NEXTN, +16%)	4,282 tok/s (MTP×3, −46%)
Best per framework	3,925 (NEXTN on)	7,886 (plain)

Table 6: Ultra-sparse MoE comparison. Both sides 100% success; R1 work pair within $\pm 0.8\%$, R2 pair 1.46% (disclosed in Section 4). ¹SGLang R1 was measured with its default CPU offload of the 51B N-gram embedding (3,296 tok/s) and with the embedding kept on GPU to match vLLM’s memory strategy (3,383 tok/s); we report the faster variant and note that the two single-run configurations differed by 2.6%.

Configuration	SGLang 0.5.20	vLLM 0.28.1rc1 (dedicated)
R1 aggregate throughput	3,669 tok/s	2,857 tok/s
R1 P95 latency	192.5 s	251.0 s
R2 (speculative, MTP)	3,135 tok/s (−15%)	2,066 tok/s (−28%, censored)
R2 success rate	100%	85.6% (ReadTimeouts)
Best per framework	3,669 (plain)	2,857 (plain)

Table 7: KDA comparison. Both R1 runs 100% success; R1 work pair 732k–734k tokens; vLLM R2 censored at 623k (Section 4).

recipe’s warning against enabling it by default on this architecture. We attribute the R1 gap to MoE execution—vLLM’s Triton MoE and FP8 pipeline for this architecture appeared more mature in the versions tested—although we did not profile kernels to confirm the attribution.

5.3 KDA hybrid: SGLang +28%

On the KDA model, SGLang again led (Table 7). The vLLM side required a *dedicated* vendor build (0.28.1rc1 with precompiled GLM kernels); the generic nightly build failed during startup with a kernel assertion and a 31-minute autotuning hang, an observation that motivated our image-governance policy of preferring dedicated builds when they exist.

Speculative decoding was a net loss on both frameworks, and on vLLM it was also a *reliability* loss: 14.4% of requests hit the client’s 300 s timeout setting, and the run ended at 301.5 s. The −28% figure is the throughput of successful responses under that timeout, and the run counted 15% fewer output tokens than its pair (623k vs 733k); the uncensored throughput difference cannot be determined from this run. The production consequence was immediate—our SGLang deployment had been running with MTP enabled since an earlier tuning round (3,057–3,135 tok/s in that configuration); disabling it raised production throughput to 3,669 tok/s (+17–20% against the 3,057–3,135 tok/s range of the MTP-on configuration), a change we shipped the same day by editing the engine’s `config.env` and restarting the engine.

5.4 Cross-architecture synthesis

Table 8 and Figure 2 collect the headline results: the leading framework changed with each model, and the margin ranged from 6% (near parity) to 2.33×.

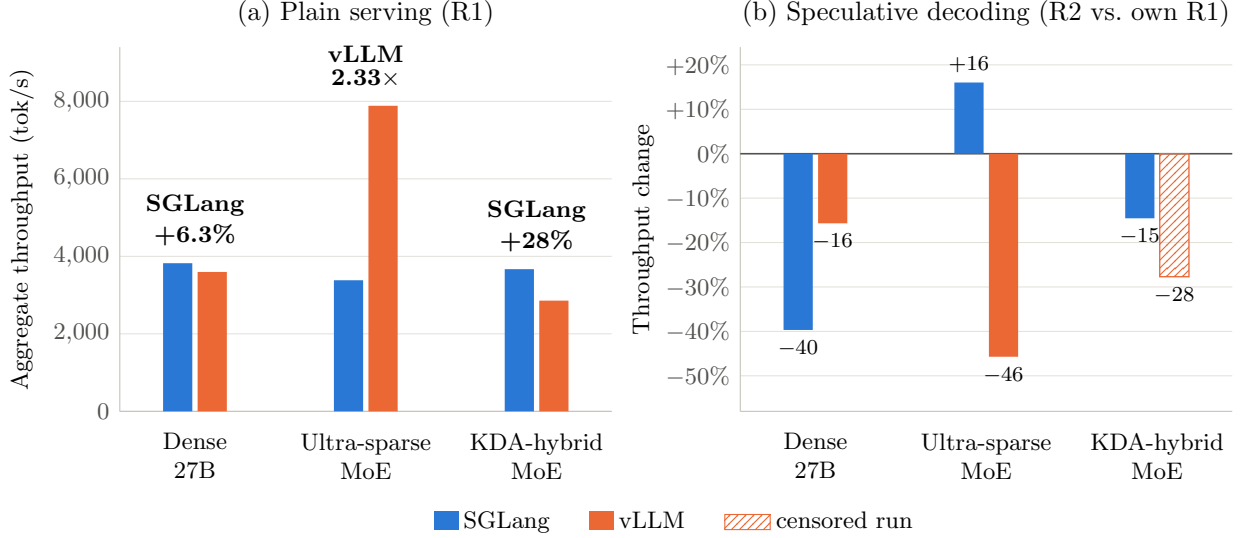


Figure 2: The three comparisons at a glance (values from Tables 4–7). (a) Plain-serving throughput; the label above each pair names the observed leader and its margin. (b) Change in aggregate throughput when the checkpoint’s speculative-decoding head is enabled, relative to the same framework’s R1; five of six bars are below zero. SGLang’s dense R2 also includes its automatic cap reduction from 128 to 48 (Section 5.1). The hatched bar is a censored measurement: 85.6% success, work truncated at 623k tokens (Section 4).

Two regularities stand out. First (Figure 2a), the framework gap was smallest on the well-trodden dense hybrid (6%) and largest on the newer ultra-sparse MoE (2.3 $\times$), which may reflect how recently each framework acquired mature kernels for a given architecture—though with one model per architecture we cannot separate this from framework-version and workload differences. Second (Figure 2b), in these runs *speculative decoding behaved as a latency tool turned throughput cost at this concurrency*: in five of the six framework–model pairs, draft-and-verify FLOPs appear not to have been recovered by acceptance gains, consistent with decode batches already binding GPU capacity (we did not measure acceptance lengths or utilization). The single positive case (SGLang NEXTN on the 6B-active MoE) is consistent with this explanation—cheap decodes leave headroom for drafting—rather than contradicting it.

5.5 Platform-level capacity and reliability

Beyond per-engine comparisons, the platform sustained a mixed workload of 1,500 concurrent users across all three engines with 100% success, 8,146 tok/s aggregate throughput, and 76.8 s P95 from the fronting VM through the tunnel and gateway (Table 9). Gateway overhead at this load was 2.5% CPU and 405 MB of memory, and no gateway-related failures were observed at the tested burst size; per-layer latency was not measured, so this does not rule out a throughput limit in the gateway.

We report this debugging arc as a result in its own right for two reasons. First, an initial hypothesis—that an 8% ReadError rate indicated a single-flow ceiling of the reverse tunnel—was *refuted* by the fix-and-remeasure cycle: after correcting both fd limits, 1,500 concurrent users traversed the single tunnel flow at 100% success, so no such ceiling was reached below 1,500. Second, the diagnosis illustrates a methodology we now apply routinely: measure each layer independently

Architecture (model)	R1 leader	Margin	R2 vs. own R1	
			SGLang	vLLM
Dense hybrid (Qwen3.8-27B)	SGLang	+6.3%	−40% ^a	−16%
Ultra-sparse MoE (Qwen3.8-Flash-Next)	vLLM	2.33×	+16%	−46%
KDA-hybrid MoE (GLM-5.3-Flash)	SGLang	+28%	−15%	−28% ^b

Table 8: Synthesis across the three comparisons. The last two columns give each framework’s throughput change when speculative decoding is enabled. ^aIncludes SGLang’s automatic cap reduction from 128 to 48 (Section 5.1). ^bCensored: 85.6% success (Section 4). Framework versions as in Table 3: SGLang 0.5.20 throughout; vLLM 0.29.1rc1 on the dense and ultra-sparse MoE models; dedicated vLLM 0.28.1rc1 build on the KDA model.

Stage of the reliability investigation	Success	Aggregate	P95	Root cause identified
Initial mixed run (1,500 users)	67.9%	—	—	Client shell <code>ulimit -n 1024</code> → fd exhaustion (Errno 24) at ~1,020 open sockets
After client fix, via tunnel	91.8%	8,251 tok/s	70.4 s	Per-connection <code>sshd</code> on the tunnel VM also capped at 1024 fds (verified via <code>/proc/<pid>/limits</code>)
Bypassing the tunnel (direct)	100%	8,156 tok/s	77.6 s	Isolated remaining losses to the VM↔gateway segment
After <code>/etc/security/limits.conf</code> fix, via tunnel	100%	8,146 tok/s	76.8 s	Correct fix: PAM <code>limits.conf</code> (systemd override alone does not apply to per-connection <code>sshd</code>); tunnel re-established after the fix

Table 9: Layered attribution of mixed-load failures (1,500 users × 500 tokens; three engines live).

(client → tunnel → gateway → engine), fix one layer at a time, and re-run the identical workload after each fix rather than trusting bypass experiments to localize faults finer than segment granularity.

6 Discussion

Framework choice is a per-model decision, and it is large. The observed spread—from a 6% margin to a 2.33× gap—is the largest lever we measured on correctly configured engines. Concurrency-cap tuning, in separate measurements on the same engines, moved throughput by 29–39% (raising the KDA engine’s cap from 64 to 128 gained 31% and lifted success from 95.7% to 100%; raising the MoE engine’s cap from 48 to 100 gained 29% on average throughput and 39% on peak); the two embedding-placement configurations differed by 2.6% in single runs; and restoring a misconfigured SGLang baseline on the dense engine had earlier been worth +48%. In operational terms, choosing the wrong framework for the ultra-sparse MoE would have cost more throughput than any misconfiguration of a correctly chosen framework. The practical implication we adopted institutionally is a *dual-framework onboarding policy*: every new model is deployed on both frameworks under suffixed aliases, A/B-tested under the production workload, and promoted by a two-line configuration change; the platform of Section 3 makes this a routine procedure rather than a redeployment project. We recommend this as default practice for multi-model serving platforms.

Speculative decoding at high concurrency. Our results add same-checkpoint evidence from a production platform to a phenomenon that goodput analyses have already identified: when speculation is applied without regard to load, its verification FLOPs compete with useful work and

acceptance gains cannot pay for them [8]. Under 500–1,000 concurrent users, with decode batches that appeared to saturate GPU capacity, this cost appeared in five of six pairs, and in the worst case the added per-request time pushed 14.4% of requests past the 300 s client timeout setting. Adaptive systems such as TurboSpec address this by gating speculation on measured goodput [8], and AdaServe by customizing it to per-request SLOs [9]; our measurements suggest a cruder but zero-engineering-cost policy—disable integrated MTP/NEXTN for short-prompt, high-concurrency throughput serving of these architectures—captures most of the benefit we observed. We do *not* conclude that speculative decoding is useless: at low concurrency, for single-stream latency, or—per MagicDec’s analysis—for long contexts under high batch [29] it can remain appropriate, and the one positive case here (SGLang NEXTN, +16% at 6B active parameters) is consistent with a model-dependent crossover, as in the goodput formulation [8], although our runs did not isolate its cause. The decision should be made from a measurement at the deployment’s own concurrency, not from the feature’s reputation.

Memory accounting can decide speculative feasibility. A secondary observation: the two frameworks’ speculative implementations have very different memory profiles on hybrid-attention models. On the dense hybrid, SGLang’s EAGLE loads the draft weights plus a 13.8 GB intermediate-state cache, and the framework automatically reduces the concurrency cap from 128 to 48 when speculative decoding is enabled (Section 5.1); vLLM’s MTP shares the head weights within the checkpoint and preserved the cap. On models whose caches dominate memory, this implementation difference alone can outweigh acceptance-rate considerations, and it is invisible in feature lists.

Dedicated builds and version governance. The KDA comparison could not be run on the generic vLLM nightly at all (kernel assertion; autotuning hang), while the vendor’s dedicated build ran correctly. Our resulting policy—version-named images that are not overwritten by default, dedicated builds preferred when they exist, “once supported $\neq$ always supported”—converted what could have been days of kernel-level debugging into a one-line image selection. We note this as a reproducibility point: framework *version* is a first-class experimental variable on new architectures, and evaluations that do not pin and report it are hard to reproduce. A version name is not a content identity, however: we did not record OCI digests or SIF checksums for the images used here, so the tested builds are identified only by name and build string (`engines/KNOWN_GOOD.md`).

What the platform measurements do and do not show. The layered analysis of Section 5.5 observed no additional gateway- or tunnel-related failures at the tested burst size: the gateway used 2.5% CPU at 1,500 users, and no single-flow tunnel ceiling was observed below 1,500 users. These runs did not pass through the TLS front end and include no capacity sweep, GPU-utilization data, or per-layer latency breakdown, so they do not establish where the next bottleneck lies or characterize the capacity of the TLS tier.

Agents as experimentalists. Beyond its findings, this study is a working example of an LLM-based agent assisting with an experimental procedure under human approval. Autonomous research agents have been demonstrated for closed-loop scientific discovery [30]; agent-driven simulation has been proposed to track the fast-moving serving design space because real deployments are costly [31]; and AutoTuneBench has agent systems auto-tune both engines under a measurement protocol stricter than ours [26]. Our pipeline is closest to the last of these but different in object: AutoTuneBench tunes engine configurations, whereas our skills drive the full production lifecycle—deployment behind a live gateway, benchmarking through the external traffic path, verdict recording, and production traffic switching—on infrastructure that keeps serving users throughout.

What made the automation trustworthy was not the agent’s raw capability but the enclosure: declarative skills that require documentation-traceable configuration, machine-checked output-token comparisons, traffic isolation that makes candidate engines harmless to production, and mandatory evidence journaling from which this paper’s tables are extracted. Within that enclosure, the agent

performed work that would otherwise gate on scarce systems-engineering time; the repository’s commit history records all three verdicts on 2026-09-27, with the vLLM counterpart engines scaffolded the same day (the SGLang engines already served production traffic), although we did not measure unattended time or labor savings; the same enclosure makes the next framework decision a documented procedure rather than an ad hoc project. The pattern extends to the manuscript itself: the literature search and citation verification (identifiers and author lists checked against the arXiv API; residual errors were caught in external review), the structural drafting from writing-methodology skills, and the evidence-alignment audit were likewise agent-performed. Related benchmarks evaluate how well agents do DevOps tasks [32]; here the DevOps task itself—deploying, benchmarking, and switching production inference infrastructure—was the experiment. We take this as an operational answer to the question this paper opened with: “which framework should serve this model?” is better answered by a measurement whose procedure is a versioned artifact of the repository than by folklore or release notes.

7 Limitations and Future Work

Three limitations bound the strength of our conclusions. First, each configuration was measured once at a single workload shape (uniform concurrency, uniform max_tokens); we did not measure variance across repeats, so no margin here is an established effect, and the dense comparison’s +6.3% should be read as near parity. The workload is a single-burst queue drain—all requests are released at once over a pool of five short prompts—so latency percentiles largely reflect queue-drain time rather than steady-state request latency, and P95 should not be read as an SLO metric. Two R2 results carry additional confounds we flag rather than resolve: SGLang’s dense R2 combines speculative decoding with the framework’s automatic cap reduction to 48 (no cap-48 plain control was run), and KDA vLLM’s R2 is censored by client timeouts (Section 4). Scheduler placement also put some A/B pairs on different nodes of the cluster. We further did not measure streaming latency—time to first token (TTFT) and time per output token (TPOT)—under open-loop arrivals, did not sweep concurrency to locate the speculative-decoding crossover, and did not record acceptance lengths or GPU utilization—so the mechanism attributions in Section 5 remain hypotheses. The harness stores per-run summary statistics only: it records no per-request lengths, finish reasons, or outputs, sets no sampling seed, and runs no output-quality check, and the prefix-cache and warm-up state of each engine was not controlled. The measurements also did not include the TLS front end, and we did not record the checkpoint revisions, tokenizer or chat-template versions, or image digests used in each run. Second, framework versions evolve monthly; our results pin SGLang 0.5.20 and vLLM 0.29.1rc1 (plus a dedicated 0.28.1rc1 build for the KDA model), and the ultra-sparse MoE gap in particular should be expected to narrow or move as kernels mature. Third, all measurements are single-engine deployments on one cluster; multi-instance load-balanced configurations were validated only for success rate (Section 5.5), not A/B-compared.

Future work should extend the matrix in six directions: repeat runs with confidence intervals and mixed workload shapes; add FP8-versus-BF16 and KV-cache-dtype axes, which our Hopper hardware partially constrains for the KDA model (FP8 KV unsupported); test whether the observed MoE framework gap closes in later SGLang releases, which would calibrate how quickly such evaluations expire; measure streaming TTFT/TPOT under open-loop arrivals; sweep concurrency to locate the speculative-decoding crossover, recording acceptance lengths; and profile GPU utilization and MoE kernel time to test the mechanism attributions offered in Section 5.

8 Conclusion

We built a multi-model API service on an HPC cluster—a LiteLLM gateway over Slurm-managed engines on H200 GPUs—and, confronted with the recurring question of which serving framework to assign to each new model, encoded the comparison procedure as versioned skills that an LLM-based agent executed under human approval: scaffold both frameworks from official documentation, deploy under traffic-isolated aliases, benchmark both under the same workload with output-token counts compared, record the verdict, and switch production traffic when warranted. Applied to three architecturally distinct LLMs, the agent-assisted comparisons found that the leading framework changed with each model—SGLang +6.3% (near parity) on a dense hybrid, vLLM by $2.33\times$ on an ultra-sparse MoE, SGLang +28% on a KDA-hybrid MoE—and that speculative decoding on the same checkpoints reduced measured throughput in five of six framework–model pairs (single-run measurements), a verdict applied to production the same day it was measured. The same agent-assisted workflow diagnosed a mixed-load reliability fault from an encoded symptom-to-cause procedure, raising success from 67.9% to 100%. Together, the results argue for treating framework selection and speculative decoding not as defaults to be inherited but as per-model, per-workload decisions—and for making the measurement that informs them cheap and safe enough to run on every new model, which is the purpose of the agent-assisted procedure described here.

AI-Use Disclosure

Research execution. The experiments reported in this paper—engine deployment, benchmarking, failure diagnosis, and production traffic switching—were executed by LLM-based coding agents following the versioned procedures (“skills”) described in Section 3, under the direction and release decisions of the human authors.

Literature search and verification. The literature search, and the verification of every arXiv identifier and author list, were performed by an agent against the arXiv API. This process had a disclosed failure mode: an intermediate reference-conversion pass introduced incorrect author lists in five entries, which were caught and corrected during independent external review—a known transcription-error class for AI-assisted literature verification. Responsibility for reference accuracy rests with the authors.

Manuscript drafting. The drafting and revision of this manuscript, including structural drafting from writing-methodology skills, evidence-alignment auditing, the LaTeX conversion, and BibTeX generation, were produced by agents from the underlying machine-recorded results; the agent used for the initial drafting and text revisions was opencode, powered by GLM-5.3. A second LLM agent, Claude Code (Anthropic Claude Opus 5.5), performed eight rounds of external review (three adversarial content reviews, three pre-submission checks, and two regression checks) whose findings are incorporated above; after the reviews, the same agent redrew Figures 1 and 2 and revised Tables 6–9, without changing any reported value. A third model, GPT-6 (ASTRA), then produced an independent external review; in response, Claude Code revised the text to narrow claims to the evidence—including the title, abstract, contributions, the benchmark path in Figure 1, and the limitations—without adding experiments or changing any reported value, and GPT-6 reviewed those revisions in two further regression rounds.

Human role. Problem framing, platform and release decisions, and verdict approval were made by the human authors, who also secured the funding. The corresponding author is responsible for final verification of the principal numerical results, calculations, and claim–citation links.

A disclosed limit of this disclosure. The experimental phases ran across multiple agent

sessions whose verbatim prompts and model versions were not preserved as timestamped transcripts; the skills, engine configurations, raw result files, job records, and commit history are released, but this disclosure cannot provide call-by-call detail. Consistent with arXiv and conference policy, no AI system is listed as an author, and the human authors take full responsibility for the manuscript’s content.

Author Contributions

Chao-Chun Chuang: Conceptualization, Methodology, Software, Investigation, Formal analysis, and Writing — original draft. **Po-Hsiang Lin:** Investigation, Data curation, Validation, and Writing — review and editing.

Funding

This work was supported in part by the National Science and Technology Council (NSTC), Taiwan, under Grant Nos. NSTC 115-2410-H-A49-046-MY3 and NSTC 114-2634-F-006-002.

Competing Interests

The authors declare that they have no competing interests.

Acknowledgments

The authors gratefully acknowledge the National Center for High-performance Computing (NCHC), National Institutes of Applied Research (NIAR), Taiwan, for providing the research resources, computational infrastructure, and platform services that supported this work.

Data and Artifact Availability

The platform repository is available at <https://github.com/gemini960114/litellm-proxy>. Raw per-run statistics (JSON) for all configurations in Tables 4–9—except the initial 67.9% mixed run, which is recorded in the platform’s operations documentation—are available in its **benchmarks/results/** directory. Engine configurations are recorded in **engines/KNOWN_GOOD.md**. The evaluation skills (model-onboarding, concurrency-troubleshooting, debug-journaling) are released with the platform repository (**.agents/skills/**). The manuscript-writing skills are third-party skills that the repository pins by content hash in **skills-lock.json** but does not redistribute. The repository is a substantial but not complete replication package: it does not redistribute the model checkpoints or the multi-gigabyte container images (referenced by version names, not content digests), and it does not preserve the agent sessions’ verbatim prompts.

References

- [1] Forough Majidi, Mohammad Mehdi Morovati, Foutse Khomh, and Heng Li. LLM serving in the wild: An empirical study of frameworks, methods, and system designs, 2026. arXiv:2608.03036.

- [2] Woosuk Kwon, Zhuohan Li, Siyuan Zhuang, Ying Sheng, Lianmin Zheng, Cody Hao Yu, Joseph E. Gonzalez, Hao Zhang, and Ion Stoica. Efficient memory management for large language model serving with PagedAttention. In *Proceedings of the 29th Symposium on Operating Systems Principles (SOSP)*, 2023. arXiv:2309.06180.
- [3] Lianmin Zheng, Liangsheng Yin, Zhiqiang Xie, Chuyue Sun, Jeff Huang, Cody Hao Yu, Shiyi Cao, Christos Kozyrakis, Ion Stoica, Joseph E. Gonzalez, Clark Barrett, and Ying Sheng. SGLang: Efficient execution of structured language model programs. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2024. arXiv:2312.07104.
- [4] Saicharan Kolluru. Comparative analysis of large language model inference serving systems: A performance study of vLLM and HuggingFace TGI, 2025. arXiv:2511.17593.
- [5] Baolin Li, Yankai Jiang, Vijay Gadepally, and Devesh Tiwari. LLM inference serving: Survey of recent advances and opportunities, 2024. arXiv:2407.12391.
- [6] David Pape, Jonathan Evertz, and Lea Schönherr. The silent hyperparameter: Quantifying the impact of inference backends on LLM reproducibility, 2026. arXiv:2605.19537.
- [7] Ranran Haoran Zhang, Aysa Xuemo Fan, David Munhá Correia, et al. SiliconBench: Speed, memory, and fidelity for LLM serving on unified-memory desktops, 2026. arXiv:2609.19169.
- [8] Xiaoxuan Liu, Jongseok Park, Langxiang Hu, Woosuk Kwon, et al. TurboSpec: Closed-loop speculation control system for optimizing LLM serving goodput, 2024. arXiv:2406.14066.
- [9] Zikun Li, Zhuofu Chen, Remi Delacourt, Gabriele Oliaro, et al. AdaServe: Accelerating multi-SLO LLM serving with SLO-customized speculative decoding, 2025. arXiv:2501.12162.
- [10] Gyeong-In Yu, Joo Seong Jeong, Geon-Woo Kim, Soojeong Kim, and Byung-Gon Chun. Orca: A distributed serving system for transformer-based generative models. In *Proceedings of the 16th USENIX Symposium on Operating Systems Design and Implementation (OSDI)*, 2022.
- [11] Amey Agrawal, Ashish Panwar, Jayashree Mohan, Nipun Kwatra, Bhargav S. Gulavani, and Ramachandran Ramjee. SARATHI: Efficient LLM inference by piggybacking decodes with chunked prefills, 2023. arXiv:2308.16369.
- [12] Yinmin Zhong, Shengyu Liu, Junda Chen, Jianbo Hu, Yibo Zhu, Xuanzhe Liu, Xin Jin, and Hao Zhang. DistServe: Disaggregating prefill and decoding for goodput-optimized large language model serving. In *Proceedings of the 18th USENIX Symposium on Operating Systems Design and Implementation (OSDI)*, 2024. arXiv:2401.09670.
- [13] Zihao Ye, Lequn Chen, Ruihang Lai, Wuwei Lin, Yineng Zhang, Stephanie Wang, Tianqi Chen, Baris Kasikci, Vinod Grover, Arvind Krishnamurthy, and Luis Ceze. FlashInfer: Efficient and customizable attention engine for LLM inference serving, 2025. arXiv:2501.01005.
- [14] An Yang, Anfeng Li, Baosong Yang, et al. Qwen3 technical report, 2025. arXiv:2505.09388.
- [15] DeepSeek-AI. DeepSeek-V3 technical report, 2024. arXiv:2412.19437.
- [16] DeepSeek-AI. DeepSeek-V2: A strong, economical, and efficient mixture-of-experts language model, 2024. arXiv:2405.04434.
- [17] GLM-5 Team, Aohan Zeng, et al. GLM-5: from vibe coding to agentic engineering, 2026. arXiv:2602.15763.

- [18] Kimi Team, Yu Zhang, Zongyu Lin, et al. Kimi Linear: An expressive, efficient attention architecture, 2025. arXiv:2510.26692.
- [19] Kimi Team, Yifan Bai, Yiping Bao, et al. Kimi K2: Open agentic intelligence, 2025. arXiv:2507.20534.
- [20] Yanqi Yu, Pingwei Sun, Jianchao Tan, et al. DASC: Decay-aware state compression for hybrid linear-attention serving, 2026. arXiv:2608.30386.
- [21] Tao Zhang, Jianchao Tan, Pingwei Sun, et al. DAMP: Decay-aware mixed-precision recurrent-state quantization, 2026. arXiv:2608.27513.
- [22] SLAM Labs, Oleksiy Ostapenko, et al. Super apriel: One checkpoint, many speeds, 2026. arXiv:2604.19877.
- [23] Yaniv Leviathan, Matan Kalman, and Yossi Matias. Fast inference from transformers via speculative decoding. In *Proceedings of the 40th International Conference on Machine Learning (ICML)*, 2023. arXiv:2211.17192.
- [24] Tianle Cai, Yuhong Li, Zhengyang Geng, Hongwu Peng, Jason D. Lee, Deming Chen, and Tri Dao. Medusa: Simple LLM inference acceleration framework with multiple decoding heads. In *Proceedings of the 41st International Conference on Machine Learning (ICML)*, 2024. arXiv:2401.10774.
- [25] Yuhui Li, Fangyun Wei, Chao Zhang, and Hongyang Zhang. EAGLE: Speculative sampling requires rethinking feature uncertainty. In *Proceedings of the 41st International Conference on Machine Learning (ICML)*, 2024. arXiv:2401.15077.
- [26] Li Chen. AutoTuneBench: Trustworthy measurement for agent auto-tuning of LLM serving engines, 2026. arXiv:2609.18123.
- [27] Keisuke Kamahori, Shihang Li, Simon Peter, and Baris Kasikci. VibeServe: Can AI agents build bespoke LLM serving systems?, 2026. arXiv:2605.06068.
- [28] Renjun Xu and Yang Yan. Agent skills for large language models: Architecture, acquisition, security, and the path forward, 2026. arXiv:2602.12430.
- [29] Ranajoy Sadhukhan, Jian Chen, Zhuoming Chen, et al. MagicDec: Breaking the latency-throughput tradeoff for long context generation with speculative decoding, 2024. arXiv:2408.11049.
- [30] Chris Lu, Cong Lu, Robert Tjarko Lange, Jakob Foerster, Jeff Clune, and David Ha. The AI scientist: Towards fully automated open-ended scientific discovery, 2024. arXiv:2408.06292.
- [31] Wonung Kim, Hyunmin Choi, Minsu Kim, Jaehong Cho, et al. Simthesizer: An agent-driven simulation framework for LLM serving systems, 2026. arXiv:2608.24650.
- [32] Yuheng Tang, Kaijie Zhu, Bonan Ruan, Chuqi Zhang, et al. DevOps-Gym: Benchmarking AI agents in software DevOps cycle, 2026. arXiv:2601.20882.